

# AsymptoticAnalysis

## Logarithmic certificate precision

Two independent losses of relative accuracy,  
native reproductions, proofs, and candidate repairs

---

|                  |                                                                                                                   |
|------------------|-------------------------------------------------------------------------------------------------------------------|
| Repository       | <a href="https://github.com/VladimirReshetnikov/Asymptotic">https://github.com/VladimirReshetnikov/Asymptotic</a> |
| Revision         | efa1aeec4845a9c35e140963a0333d0c9ec33b05                                                                          |
| Review date      | 10 September 2026                                                                                                 |
| Native evidence  | Wolfram Language 15.0.0, Linux x86-64; downloaded standalone                                                      |
| Other evidence   | Exact-rational Python models; 24 passing test methods                                                             |
| Mathics boundary | Source analysis and supplied regressions; no Mathics execution                                                    |

---

**Principal result.** Two distinct implementation choices discard relative accuracy around a logarithm's unit argument. The baseline safely refuses three exact-root certificate requests. Two small, independently tested changes make all three requests certify at the same enclosure order. No false certificate or incorrect expansion is alleged by these findings.

**Novelty boundary.** This is an incremental review after six existing review waves. Known findings and their general roadmaps are excluded from the new work. The retained contribution is deliberately narrower than a claim of exhaustive verification of every repository file.

The archive contains the article source, candidate emitter, reproduction specifications, independent models, selected native observations, and explicit evidence limitations. Candidate code is not an upstream change.

## Abstract

This review investigates the current AsymptoticAnalysis implementation against its Wolfram and Mathics compatibility objective while screening out obligations already recorded in the repository's external reviews. Two new shortcomings remain after that screening. First, a positive rational just below one is reduced to a logarithm near two minus a separately enclosed logarithm of two; the resulting interval has an absolute uncertainty essentially unrelated to the tiny logarithm being computed. Second, an exactly computed rational affine argument of a logarithm is rounded near one before the logarithm is evaluated, losing the small displacement that determines the answer. These mechanisms are distinct: a four-variant, three-witness native experiment separates them.

The proposed repairs use reciprocal reduction below one and preserve exact affine endpoints until after taking the logarithm. Their containment arguments are proved, their selected public consequences are reproduced on Wolfram 15.0.0, and independent rational models pass 24 test methods. The article also develops a narrowly scoped, positivity-guarded logarithmic-ratio residual prototype. Mathics execution, canonical-source integration, a complete MUnit run, and performance benchmarking remain outside the executed evidence.

## Contents

|          |                                                                 |          |
|----------|-----------------------------------------------------------------|----------|
| <b>1</b> | <b>Executive assessment</b>                                     | <b>3</b> |
| 1.1      | What should change . . . . .                                    | 3        |
| 1.2      | What this result does not establish . . . . .                   | 3        |
| <b>2</b> | <b>Scope, provenance, and nonduplication</b>                    | <b>4</b> |
| 2.1      | Pinned source and execution boundary . . . . .                  | 4        |
| 2.2      | Breadth of inspection . . . . .                                 | 4        |
| 2.3      | How existing findings were excluded . . . . .                   | 5        |
| <b>3</b> | <b>The certificate arithmetic relevant to the findings</b>      | <b>5</b> |
| 3.1      | Exact intervals and directed significant-bit rounding . . . . . | 5        |
| 3.2      | The existing logarithm formula . . . . .                        | 6        |
| 3.3      | Why a wide residual prevents certification . . . . .            | 6        |
| <b>4</b> | <b>F01: near-one cancellation in the point logarithm</b>        | <b>7</b> |
| 4.1      | Source mechanism . . . . .                                      | 7        |
| 4.2      | Native witness A . . . . .                                      | 8        |
| 4.3      | Candidate repair and inclusion proof . . . . .                  | 8        |
| 4.4      | Executed effect . . . . .                                       | 9        |
| <b>5</b> | <b>F02: rounding an affine argument before its logarithm</b>    | <b>9</b> |
| 5.1      | Where correct exact work is discarded . . . . .                 | 9        |
| 5.2      | Native witness B . . . . .                                      | 10       |

|           |                                                                    |           |
|-----------|--------------------------------------------------------------------|-----------|
| 5.3       | Candidate repair . . . . .                                         | 10        |
| 5.4       | Executed effect and costs . . . . .                                | 11        |
| <b>6</b>  | <b>Independent causes: the controlled native comparison</b>        | <b>11</b> |
| 6.1       | A translated negative-side witness . . . . .                       | 11        |
| 6.2       | Four full standalone variants . . . . .                            | 11        |
| 6.3       | Negative and fallback controls . . . . .                           | 12        |
| 6.4       | Conditioning versus representation . . . . .                       | 12        |
| <b>7</b>  | <b>Quantifying the avoidable work</b>                              | <b>13</b> |
| 7.1       | A rational work model, not a benchmark . . . . .                   | 13        |
| 7.2       | Representative exact-model widths . . . . .                        | 14        |
| <b>8</b>  | <b>Wolfram 15 and Mathics3 implications</b>                        | <b>14</b> |
| 8.1       | What was verified on Wolfram . . . . .                             | 14        |
| 8.2       | Why the source finding matters to Mathics . . . . .                | 14        |
| 8.3       | A focused cross-runtime acceptance obligation . . . . .            | 15        |
| <b>9</b>  | <b>E01: center-preserving logarithmic residuals</b>                | <b>15</b> |
| 9.1       | A narrower next step than a general expression optimizer . . . . . | 15        |
| 9.2       | Executed primitive, unintegrated proposal . . . . .                | 16        |
| 9.3       | Required guardrails . . . . .                                      | 16        |
| <b>10</b> | <b>Tests, artifacts, and reproducibility</b>                       | <b>16</b> |
| 10.1      | Independent test population . . . . .                              | 16        |
| 10.2      | Candidate emitter and application discipline . . . . .             | 17        |
| 10.3      | Native and portable reproduction . . . . .                         | 17        |
| 10.4      | Evidence inventory . . . . .                                       | 18        |
| <b>11</b> | <b>Recommended integration sequence and conclusion</b>             | <b>18</b> |
| 11.1      | Immediate scoped work . . . . .                                    | 18        |
| 11.2      | Final assessment . . . . .                                         | 19        |

# 1 Executive assessment

## 1.1 What should change

The highest-value new work found in this audit is concentrated in `src/Kernel/InverseCertificates.wl`. The certificate engine is already careful about outward rounding, real domains, derivative separation, and existence evidence. The problems below are not failures of those safety checks. They make otherwise straightforward requests consume unnecessary precision or fail to produce a certificate at the requested resource settings. The inspected source is the pinned revision, not an inferred current branch state.[\[1, 2\]](#)

| ID  | New obligation                                                                                                                                                                  | Evidence and priority                                                             |
|-----|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------|
| F01 | Preserve relative precision when evaluating an exact rational logarithm just below one. Replace cancellation of two near-equal log enclosures by a single reciprocal reduction. | Native reproduction and repair; functional robustness/performance priority.       |
| F02 | Do not round an exact affine logarithm argument near one before applying the logarithm. Pass its exact range to the logarithm primitive.                                        | Native reproduction and repair; functional robustness/performance priority.       |
| E01 | Explore a positive logarithmic-ratio residual that retains an exact zero at a known rational center.                                                                            | New scoped development proposal; primitive checked, dispatcher integration unrun. |

Table 1: Retained deltas. F01 and F02 are safe-refusal shortcomings, not unsound certificates. E01 is not counted as a demonstrated defect.

For  $\delta = 2^{-200}$ , the original low-order point enclosure of  $\log(1-\delta)$  straddles zero with endpoint magnitude approximately  $1.85 \times 10^{-3}$ , although the logarithm has magnitude about  $6.22 \times 10^{-61}$ . Separately, evaluating  $\log(1+x)$  at the exact point  $x = \delta$  produces an enclosure approximately  $[0, 7.11 \times 10^{-15}]$ . Both are valid enclosures; both are too wide for the deliberately small verification intervals in the public witnesses.

The combined candidate certifies all three native witnesses at "EnclosureOrder"  $\rightarrow$  2 and "MaxRefinements"  $\rightarrow$  0. In every successful row of the controlled comparison, the returned root interval contains the known exact root. The corresponding bounds on error of the *supplied center* are about  $4.42 \times 10^{-75}$  or  $2.21 \times 10^{-75}$ . They are not automatically error bounds for the stored finite asymptotic expression.

## 1.2 What this result does not establish

An enclosure-order option is a resource choice, not a promise that every problem must certify at that order. Thus a baseline refusal alone would be weak evidence of a bug. The stronger evidence is the identified loss mechanism, a family showing its scaling, a containment-preserving repair, and the same-resource native comparison. These justify treating the behavior as an avoidable numerical and coverage shortcoming without portraying a safe failure as an incorrect theorem.

The successful candidate observations do not establish complete Wolfram compatibility, complete Mathics compatibility, or general superiority in every logarithm calculation. No default-setting benchmark was run. In particular, the report does not claim that ordinary adaptive defaults always

fail for these moderate-size examples: increasing arithmetic work can recover some lost information. The issue is that substantially more work is then demanded for an avoidable representation choice.

## 2 Scope, provenance, and nonduplication

### 2.1 Pinned source and execution boundary

The reviewed commit is `efa1aeec4845a9c35e140963a0333d0c9ec33b05`, committed on 10 September 2026 at 19:09:04 UTC. Its root standalone package contains 692,381 bytes. The remote native experiments downloaded this exact standalone to a temporary file and loaded it into a Wolfram Language kernel identifying itself as `15.0.0 for Linux x86 (64-bit) (May 6, 2026)`. The four-variant comparison reloaded temporary standalone copies within one kernel; it is not four independent environments. Other focused native probes were separate calls.[1]

A local repository clone and a local Mathics installation could not be obtained because the local environment's network access failed. The GitHub connector supplied source text, and the remote Wolfram evaluator supplied selected package executions. The remote service was intermittent: failed service calls are not counted as package failures, successful tests, or completed corpus scans.

The following distinctions apply throughout:

#### Native observation

A reported value came from a successful Wolfram call loading the pinned standalone or an explicitly identified temporary candidate.

#### Independent model

Python's `Fraction` arithmetic implements the small enclosure mechanism. It does not execute the repository and is not a Mathics simulation claim.

#### Proof

A mathematical argument establishes an enclosure or scaling property under stated exact-real hypotheses. It is not a machine-checked formalization.

#### Regression specification

A supplied WL/WLT file records desired behavior. Its existence is not evidence that the complete file or MUnit suite was executed.

No upstream files were modified. No canonical modular build, standalone freshness check, full package suite, complete new portable suite, or wall-clock performance benchmark was run. The code in the archive is a candidate and a reproduction aid, not a replacement release.

### 2.2 Breadth of inspection

The source inspection extended beyond the two retained findings. It covered public request and result definitions; substantial parts of the forward power-log jet arithmetic; the fixed-parameter special-function adapter; exact special-function identity normalization; real-domain guards; envelope arithmetic; selected Gamma, Dirichlet, and inverse-check paths; Mathics assumption, numerical, list, and compatibility adapters; and most of the certificate module. The fixed-parameter and identity modules were inspected particularly for lost branches, incorrect offsets, and finite identities mistaken for asymptotic truncations. No additional new defect in those paths was established by the performed analysis.[3, 4, 5]

This is not a claim to have read every module line by line or to have verified all formulas reachable through native special functions. A broad search followed by a small retained set is preferable to padding the report with already-known or unverified defects. The useful deliverable here is a precise incremental contribution with runnable evidence.

## 2.3 How existing findings were excluded

The repository index records 45 retained packages across six review waves, with ten retired packages. Its count of 231 entries for waves 1–4 is a count of attributed report entries before consolidation, not 231 distinct current bugs. This audit used all six wave indexes and substantial portions of the maintained implementation register to screen candidate findings.[6, 7, 8, 9]

A targeted additional text screen fetched twelve retained wave-2 TeX files and searched for the certificate logarithm helper names and near-one/log1p terms. That completed scan had no fetch failures and no matches. Larger attempted scans failed and are not claimed as completed. Not every retained article or PDF was individually reread. Consequently the defensible novelty claim is that these two mechanisms are absent from the inspected maintained crosswalks and targeted material, not a logically exhaustive proof that no sentence in the entire archive could be related.

| Existing item | Previously recorded concern                                                                                | Why the retained delta differs                                                                                                                       |
|---------------|------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------|
| C19           | Certificate accuracy planning, progress, tolerances, and refinement.                                       | F01 and F02 occur inside a single exact arithmetic attempt. They remain separable with adaptation disabled.                                          |
| C20           | Preserve cancellation when finding an affine range, especially with a large translated source coordinate.  | F02 starts after that exact range has already been computed correctly. The later rounding before a logarithm loses a different small quantity.       |
| P08           | General profiling of proof and arithmetic work.                                                            | The new report gives a specific operation, a parameterized loss mechanism, and executed local repairs, not another generic profiling recommendation. |
| Wave 6        | Mathics logarithm splitting authorized by rounded sign tests; interval-power geometry and budget behavior. | The new witnesses use the shared rational logarithm enclosure with positive exact inputs. No machine-sign split or interval power is involved.       |

Table 2: Nearest existing obligations are acknowledged, not restated as new findings. The archive contains a separate novelty crosswalk.

The existing generic capability, native-backend, multiscale, certification, release, and provenance roadmaps are not repeated as new recommendations. The implementation plan below is restricted to the two repairs and their centered-residual extension.

## 3 The certificate arithmetic relevant to the findings

### 3.1 Exact intervals and directed significant-bit rounding

Let an interval  $[a, b]$  with rational endpoints represent a proved real inclusion. For a nonzero rational  $q$ , the code chooses

$$e = \lfloor \log_2 |q| \rfloor, \quad h = 2^{e-B+1},$$

where  $B$  is the significant-bit budget. It rounds downward with  $h\lfloor q/h \rfloor$  and upward with  $h\lceil q/h \rceil$ . Addition and multiplication compute exact rational endpoint expressions, then round outward. No floating-point sign test is used in this primitive.<sup>[2]</sup>

This arrangement protects inclusion. It does not retain dependence between separately enclosed occurrences of almost the same quantity. Nor does exact rational evaluation prior to one rounding step guarantee that a later ill-conditioned transformation will preserve useful relative accuracy. Those are the respective sources of F01 and F02.

The public driver sets

$$B = 4(n + 10), \quad (1)$$

where  $n$  is "EnclosureOrder". It accepts  $2 \leq n \leq 2000$ . The main witnesses intentionally use  $n = 2$ , hence  $B = 48$ , and zero refinements. They give an exact rational center strictly inside an exact rational interval, so the numerical seed path is not the cause of failure.

### 3.2 The existing logarithm formula

For  $1 \leq q \leq 2$ , set

$$u = \frac{q-1}{q+1}, \quad 0 \leq u \leq \frac{1}{3}.$$

The classical identity

$$\log q = 2 \sum_{k=0}^{\infty} \frac{u^{2k+1}}{2k+1} \quad (2)$$

is the logarithmic form of the inverse hyperbolic tangent series.<sup>[11]</sup> The code retains  $n$  terms and bounds the remaining positive tail:

$$S_n(q) = 2 \sum_{k=0}^{n-1} \frac{u^{2k+1}}{2k+1}, \quad (3)$$

$$T_n(q) = \frac{2u^{2n+1}}{(2n+1)(1-u^2)}. \quad (4)$$

Indeed, for  $k \geq n$  the denominator  $2k+1$  is at least  $2n+1$ , so the remaining series is bounded by a geometric series. Thus

$$S_n(q) \leq \log q \leq S_n(q) + T_n(q). \quad (5)$$

The final outward rounding preserves this enclosure.

For a general positive rational, the original point routine takes  $e = \lfloor \log_2 q \rfloor$ ,  $z = q/2^e \in [1, 2)$ , and combines independent enclosures using

$$\log q = \log z + e \log 2. \quad (6)$$

The identity is correct. Its interval evaluation is the problem when the answer is a tiny difference of two quantities close to  $\log 2$ .

### 3.3 Why a wide residual prevents certification

Suppose the verification interval is  $I = [a, b]$ , the candidate center is  $c \in (a, b)$ , the stored forward function is continuously differentiable on  $I$ , and the derivative has a fixed sign with  $|f'| \geq \mu > 0$  there. If the residual enclosure proves  $|f(c) - y| \leq \varepsilon$ , the natural residual radius is  $\varepsilon/\mu$ .

When the resulting symmetric bracket is contained in  $I$ , continuity and the derivative sign establish a root in that bracket. To see this in the increasing case, use

$$f(c - r) - y \leq f(c) - y - \mu r \leq 0, \quad f(c + r) - y \geq f(c) - y + \mu r \geq 0,$$

with  $r = \varepsilon/\mu$ . The decreasing case is analogous. The code can also establish existence through separately enclosed endpoint residual signs. After existence is known, its signed derivative enclosure sharpens the root interval.[2]

A residual bound of size  $10^{-3}$  or  $10^{-15}$  cannot establish the desired local bracket inside an interval of width about  $10^{-60}$  when the endpoint residuals are similarly overestimated. Refusing the certificate is then correct. Improving the arithmetic that supplied those bounds, rather than weakening existence or derivative checks, is the appropriate repair.

## 4 F01: near-one cancellation in the point logarithm

### 4.1 Source mechanism

The relevant routine is `certLogPoint`, at canonical source lines 61–72. After rejecting nonpositive arguments and returning an exact zero for one, it performs the binary range reduction in (6). There is no special treatment of a rational below one.[2]

Let  $q = 1 - \delta$  with  $0 < \delta < 1/2$ . Then  $e = -1$ , so the implemented calculation is

$$\log(1 - \delta) = \log(2 - 2\delta) - \log 2. \quad (7)$$

The intervals for the two logarithms are treated as independent, although they approach the same value as  $\delta \rightarrow 0^+$ .

**Proposition 4.1** (Fixed-order loss). *Before outward rounding, the interval produced by the two unit-log bounds in (7) tends to*

$$[-T_n(2), T_n(2)] \quad \text{as } \delta \rightarrow 0^+,$$

where

$$T_n(2) = \frac{3^{1-2n}}{4(2n+1)} > 0. \quad (8)$$

For every fixed  $n$ , sufficiently small positive  $\delta$  therefore makes this enclosure straddle zero. Outward rounding cannot repair the sign loss.

*Proof.* The lower and upper endpoints of the unrounded subtraction are

$$S_n(2 - 2\delta) - S_n(2) - T_n(2)$$

and

$$S_n(2 - 2\delta) + T_n(2 - 2\delta) - S_n(2).$$

Continuity of these finite rational expressions near two gives the stated limits. Substituting  $u = 1/3$  into (4) gives (8). Since the limiting lower endpoint is strictly negative and the limiting upper endpoint strictly positive, the crossing persists for sufficiently small  $\delta$ . Enlarging an interval by outward rounding cannot remove that crossing.  $\square$

For  $n = 2$ ,  $T_2(2) = 1/540$ . The problem is not an algebraic identity error or inexact input: it remains in exact rational arithmetic with valid explicit tail bounds. At  $q = 1$  itself the exact-zero branch is used; the precision behavior is sharply different immediately below it.



## 4.2 Native witness A

The following is the mathematical core of the executed request. Load the pinned standalone in a separate input expression first; the archive's portable probe uses fully qualified package symbols.

```
d = 2^-200;
s = AsymptoticInverse[Log[x], {x, 1}, {y, 3},
  Direction -> "FromBelow"];
r = InverseCertificate[s, Log[1-d],
  "Interval" -> {1-2d, 1-d/2}, "Center" -> 1-d,
  "EnclosureOrder" -> 2, "MaxRefinements" -> 0];
```

The exact root is  $x_* = 1 - d$ . The interval lies strictly below the expansion point one and contains that root in its interior. On the interval,  $f'(x) = 1/x > 1$ . The native failure reported a derivative lower bound of one, so failure to separate the derivative from zero is not involved.

The baseline returned `Failure["ResidualBracketOutsideInterval", ...]`. A separate call to the point logarithm primitive at the same order displayed approximately

$$[-0.00185185185185, 0.00185185185185].$$

That primitive's full exact interval was not returned in the successful native transcript. The independent rational model gives the exact endpoints

$$\pm \frac{65156244609}{3518437208832},$$

whose numerical values agree with the displayed native result. These exact endpoints are model evidence, not an invented exact native transcript.

The root was supplied exactly, and the symbolic target was `Log[1-d]`. No reference-root numerical solver, approximate target, or unproved asymptotic error constant is needed to explain the failure.

## 4.3 Candidate repair and inclusion proof

Insert the following line immediately after the existing exact-one case, with the existing positivity check retained:

```
If[q < 1,
  Return[certNeg[certLogPoint[1/q, ctx]], Module]];
```

All arguments to the added branch are positive exact rationals. If  $q < 1$ , then  $1/q > 1$ , so recursion occurs at most once. Arguments at least one continue through the original branch, including the exact result at one.

**Proposition 4.2** (Reciprocal repair). *If the original positive-input logarithm routine returns a containing interval  $[L, U]$  at  $1/q$ , then the candidate interval  $[-U, -L]$  contains  $\log q$ . For  $q = 1 - \delta$  and  $0 < \delta < 1/2$ , the new unit-series variable is*

$$u' = \frac{\delta}{2 - \delta}. \tag{9}$$

*At fixed  $n$ , its unrounded tail is  $O(\delta^{2n+1})$ .*

*Proof.* The positive-real identity  $\log q = -\log(1/q)$  reverses the interval endpoints and proves inclusion. Since  $1 < 1/(1 - \delta) < 2$ , the binary exponent of the reciprocal is zero. Substitution into  $(z - 1)/(z + 1)$  gives (9). Equation (4) then gives the stated tail order.  $\square$

Because  $|\log(1 - \delta)| \sim \delta$ , the relative tail is  $O(\delta^{2^n})$ . Outward rounding of the resulting small logarithm has absolute scale  $O(\delta 2^{-B})$ , rather than the scale of a number near  $\log 2$ . This explains why the repair helps even though it does not increase the significant-bit budget.

The proof establishes containment, not a claim that the new interval is narrower than the original for every positive rational far from one. Nor does it assert constant bit complexity: representing an input such as  $1 - 2^{-m}$  already requires a number of bits growing with  $m$ .

## 4.4 Executed effect

The patch anchor matched the downloaded standalone exactly once. Loading a temporary copy with the inserted line changed witness A to a successful certificate. The returned root enclosure contained  $1 - 2^{-200}$ , and its bound on error of the supplied center was approximately  $4.42 \times 10^{-75}$ . The primitive's observed width divided by  $2^{-200}$  was about  $7.11 \times 10^{-15}$ ; the exact independent model value is  $2^{-47}$ .

This is a controlled improvement with the same input interval, center, enclosure order, and zero refinement budget. It is not an acceptance run of all certificates or a measurement of end-to-end speed.

# 5 F02: rounding an affine argument before its logarithm

## 5.1 Where correct exact work is discarded

The package already contains `certAffinePair` and `certAffineRange`. They recognize rational affine trees built from constants, the source symbol, `Plus`, and `Times`, and compute an exact range. That existing mechanism is not missing and is not proposed again here.[2, 7]

However, `certEnclose` rounds that exact affine range before returning it. The fallback in `certLogExpression` then applies `certLog` to the rounded argument. The loss occurs in the sequence

$$\text{exact affine range} \longrightarrow \text{rounded range near 1} \longrightarrow \text{logarithm enclosure.} \quad (10)$$

The relevant source is the logarithm-expression helper at lines 80–94 and the affine enclosure path at lines 109–130.

For  $x = \delta = 2^{-200}$ , the exact value of  $1 + x$  is available. At  $B = 48$  its significant-bit grid near one has spacing  $2^{-47}$ , so outward rounding turns the point into  $[1, 1 + 2^{-47}]$ . Taking a logarithm of that interval must include zero. Its width is governed by the precision near one, not by the size of  $\log(1 + \delta)$ .

**Proposition 5.1** (Information lost before a logarithm). *Let  $B \geq 2$ ,  $h = 2^{1-B}$ , and  $0 < \delta < h$ . The significant-bit outward rounding of the singleton  $1 + \delta$  is  $[1, 1 + h]$ . Every sound enclosure of the logarithm over that rounded interval contains both 0 and  $\log(1 + h)$ . Increasing only the accuracy of the subsequent logarithm calculation cannot recover the original singleton's information.*

*Proof.* All values in  $[1, 2)$  use exponent zero and grid  $h$ . For  $0 < \delta < h$ , the downward grid neighbor is one and the upward grid neighbor is  $1 + h$ . The logarithm is strictly increasing on the positive real axis, and its exact image of that rounded interval is  $[0, \log(1 + h)]$ . Any sound enclosing interval must contain this image, irrespective of how accurately its endpoints are computed.  $\square$

This is distinct from F01. For an exact point just above one, the original point logarithm has binary exponent zero and does not subtract a logarithm of two. The loss in F02 happens before that otherwise favorable primitive receives its input.

## 5.2 Native witness B

```
d = 2^-200;
s = AsymptoticInverse[Log[1+x], {x, 0}, {y, 3}];
r = InverseCertificate[s, Log[1+d],
  "Interval" -> {d/2, 2d}, "Center" -> d,
  "EnclosureOrder" -> 2, "MaxRefinements" -> 0];
```

Here the exact inverse is  $e^y - 1$ , the supplied root is  $x_* = d$ , and the package's retained finite expression was  $y + y^2/2$ . On the given positive interval,  $f'(x) = 1/(1+x)$  is positive and close to one.

The baseline again returned `Failure["ResidualBracketOutsideInterval", ...]`. Its recorded residual enclosure was

$$[-2^{-200}, 2^{-47}],$$

and its derivative lower bound was the positive rational

$$\frac{140737488355327}{140737488355328}.$$

A primitive call displayed the logarithm enclosure  $[0, 7.105427357601 \times 10^{-15}]$  at the exact singleton source point. The interval is conservative, not incorrect, but its uncertainty is much larger than the verification interval.

A separate decimal display of `N[Log[1+2^-200], 12]` in that native batch emitted `N:meprec`. That display is not used as a twelve-digit oracle in this article. The package received an exact symbolic target, and the independent numerical controls use `mpmath.log1p` at sufficient precision. Confusing a display warning with an exact-certificate failure would obscure the actual mechanism.

## 5.3 Candidate repair

Before the existing special cases in `certLogExpression`, add:

```
If[MemberQ[{Plus, Times}, Head[argument]],
  candidate = certAffineRange[argument, x, interval];
  If[ListQ[candidate],
    Return[certLog[candidate, ctx], Module]]];
```

The existing `candidate` local is reused. The change does not ask for a new general symbolic solver. It consumes the exact range already supported by the package and delays rounding until the logarithm primitive has converted the near-unit argument into the small answer.

**Proposition 5.2** (Exact affine logarithm range). *Let  $p(x) = ax + b$  with  $a, b \in \mathbb{Q}$ , let  $I = [\ell, r]$  have rational endpoints, and let the exact range  $J = p(I)$  be strictly positive. If the point-log routine encloses the logarithm of each endpoint of  $J$ , choosing its lower bound at  $\min J$  and upper bound at  $\max J$  encloses  $\log p(x)$  for every  $x \in I$ .*

*Proof.* An affine function attains its extrema at interval endpoints; sorting  $a\ell + b$  and  $ar + b$  therefore gives its exact range, including when  $a < 0$ . The logarithm is increasing on  $(0, \infty)$ . Its range is

consequently  $[\log \min J, \log \max J]$ , which is enclosed by the specified point bounds. The existing `certLog` check rejects a nonpositive lower endpoint, so the repair does not discard the required domain condition.  $\square$

For a nonaffine argument, the recognizer returns failure and the original fallback remains available. For a positive product whose individual factors are negative, an exact affine range is sufficient: the repair does not need to assert real logarithms of those individual negative factors. This is not an unrestricted product-logarithm rewrite.

## 5.4 Executed effect and costs

The F02 anchor also matched exactly once in the full standalone. With this repair alone, witness B certified, its exact root was contained, and the center error bound was approximately  $2.21 \times 10^{-75}$ . The primitive's width divided by  $2^{-200}$  was about  $3.55 \times 10^{-15}$ , with exact independent model value  $2^{-48}$ .

For admitted affine expressions, exact endpoint construction was already performed by the old path; the candidate avoids throwing away that information before the logarithm. For a nonaffine `Plus` or `Times` argument, the new dispatch can perform an extra unsuccessful affine probe before the old path. Thus the repair is not advertised as a universal speedup. A future implementation should measure this specific additional work while retaining the mathematical invariant, rather than changing unrelated affine recognizer architecture as part of this patch.

# 6 Independent causes: the controlled native comparison

## 6.1 A translated negative-side witness

The third witness is

```
d = 2^-200;
s = AsymptoticInverse[Log[1+x], {x, 0}, {y, 3},
  Direction -> "FromBelow"];
r = InverseCertificate[s, Log[1-d],
  "Interval" -> {-2d, -d/2}, "Center" -> -d,
  "EnclosureOrder" -> 2, "MaxRefinements" -> 0];
```

Call this witness C. It is the same equation as A after the exact coordinate translation  $x_A = 1 + x_C$ . The source interval and center are translated with it. However, the expression-tree path now computes the affine argument  $1 + x_C$ , so both early rounding and below-one reduction can matter.

This is an especially useful regression because it does not rely on comparing unrelated mathematical functions. A source coordinate change should not manufacture orders of magnitude of avoidable proof uncertainty.

## 6.2 Four full standalone variants

The controlled experiment loaded the baseline and three temporary source variants: reciprocal only, exact-affine logarithm only, and both. It evaluated A, B, and C under each variant. The complete matrix is stored in `evidence/native-matrix.json`; it is a selected transcription of a successful remote response, not a raw MUnit receipt.

| Source variant | A: $\log x$ below 1 | B: $\log(1 + x)$ , $x > 0$ | C: $\log(1 + x)$ , $x < 0$ |
|----------------|---------------------|----------------------------|----------------------------|
| Baseline       | Refused             | Refused                    | Refused                    |
| F01 only       | Certified           | Refused                    | Refused                    |
| F02 only       | Refused             | Certified                  | Refused                    |
| F01 and F02    | Certified           | Certified                  | Certified                  |

Table 3: Twelve native public evaluations. Every refusal had tag `ResidualBracketOutsideInterval`; every certification contained the known exact root. The enclosure order and refinement budget were unchanged.

The matrix establishes noninterchangeability. F01 alone does not recover the small affine displacement in B. F02 alone does not cure the below-one point reduction in A. For C, resolving only one loss leaves the other active. The combined result therefore supplies stronger evidence than testing each patch only on its own favorable example.

The series constructions in the matrix were not regenerated for every variant; the changes target certificate arithmetic, and the same stored inputs were used. Reloading definitions in one kernel is useful for a controlled comparison but does not substitute for clean-kernel installation and modular loading checks before integration. Separate preliminary calls nevertheless also reproduced the individual baseline and candidate mechanisms.

### 6.3 Negative and fallback controls

Additional executed native controls compared the baseline and combined candidate. The nonlinear argument  $1+x^2$  on  $\{1,2\}$  produced an identical enclosure. The positive argument  $-x$  on  $\{-2,-1\}$  also produced an identical result. A logarithm of  $1+x$  over  $\{-2,0\}$  retained an `IntervalDomain` refusal, as did a point logarithm at zero. The value at one remained exactly  $\{0,0\}$ .

These controls address concrete risks of the edits: bypassing a domain check, breaking the nonlinear fallback, or mishandling negative factors of a positive argument. They are not an exhaustive equivalence check of all old and new interval widths. A separate suspected lookup-default issue was dismissed by a native control showing the expected shared lazy default; it is not included as a third finding.

### 6.4 Conditioning versus representation

It would be inaccurate to say that  $q \mapsto \log q$  is relatively well-conditioned near one: its relative condition number is  $1/|\log q|$ . The audit concerns an *exact* rational  $q$  whose small displacement is known. With that displacement as input, the maps

$$\delta \mapsto \log(1 + \delta), \quad \delta \mapsto \log(1 - \delta)$$

have relative condition numbers tending to one as  $\delta \rightarrow 0$ . The corresponding inverse-root derivatives in the witnesses are also close to one. The excessive uncertainty arises from changing that exact representation into separately rounded or separately enclosed large terms.

A robust certificate engine should exploit exact structure when it is available, while remaining conservative when it is not. The proposed edits follow that principle without weakening any certified inclusion.

## 7 Quantifying the avoidable work

### 7.1 A rational work model, not a benchmark

The archive includes exact models of the rounded primitive and a separate order-cost table. These calculations do not measure wall time, peak memory, actual adaptive iterations, or total package cost. They show how two known sources of uncertainty scale with  $\delta = 2^{-m}$ .

For F01, define a simple comparison threshold by requiring

$$T_n(2) \leq \frac{2^{-m}}{8}. \quad (11)$$

The smallest such  $n \geq 2$  is an exact rational work proxy for reducing the near-log 2 tail to a fraction of the small target scale. It is not a necessary and sufficient certificate-success condition. The actual expression also contains the second tail, outward rounding, derivative bounds, and interval geometry.

For F02, avoiding the particular rounded point interval  $[1, 1 + 2^{1-B}]$  at  $1 + 2^{-m}$  requires at least

$$B \geq m + 1.$$

With the driver's rule (1), this gives the lower bound

$$n \geq \max\left(2, \left\lceil \frac{m+1}{4} \right\rceil - 10\right). \quad (12)$$

This lower bound only removes the identified grid loss; it does not prove that all remaining certificate checks will succeed.

| $m$ in $\delta = 2^{-m}$ | Least $n$ satisfying (11) | Grid lower bound (12) |
|--------------------------|---------------------------|-----------------------|
| 100                      | 31                        | 16                    |
| 200                      | 62                        | 41                    |
| 500                      | 156                       | 116                   |
| 1000                     | 314                       | 241                   |
| 10000                    | 3152                      | 2491                  |

Table 4: Exact arithmetic work models, not kernel timings or measured retry counts. The last row is a theoretical scale example, not an executed 10,000-bit public stress test.

Since the driver caps  $n$  at 2000, its largest  $B$  is 8040. At  $m = 10000$ , early rounding of  $1 + 2^{-m}$  still loses the small displacement even at that cap. This identifies a precision obstacle in the original path, not a newly executed cap-level failure. The narrow candidate preserves the exact endpoints instead of attempting to hide the obstacle by raising a global cap.

For F01, (11) implies an order growing asymptotically like  $m/(2 \log_2 3)$  up to logarithmic corrections. With reciprocal reduction, the series variable itself is of order  $2^{-m}$ , so a fixed number of terms can already make the tail negligible relative to a fixed significant-bit rounding budget.

## 7.2 Representative exact-model widths

At  $m = 200$ ,  $n = 2$ , and  $B = 48$ , the independent model gives

$$\begin{aligned}\frac{\text{wid}(\mathcal{I}_{\text{original}}(\log(1 - \delta)))}{\delta} &\approx 5.95 \times 10^{57}, \\ \frac{\text{wid}(\mathcal{I}_{\text{reciprocal}}(\log(1 - \delta)))}{\delta} &= 2^{-47}, \\ \frac{\text{wid}(\mathcal{I}_{\text{early\_round}}(\log(1 + \delta)))}{\delta} &\approx 1.14 \times 10^{46}, \\ \frac{\text{wid}(\mathcal{I}_{\text{affine\_fused}}(\log(1 + \delta)))}{\delta} &= 2^{-48}.\end{aligned}$$

The CSV includes further scales from  $m = 20$  to  $m = 1000$ . At smaller  $m$ , the affine input may already be adequately representable, and F02 need not improve the result. This is another reason not to advertise a universal improvement factor.

The exact rational arithmetic still has input-size and intermediate-size costs. In particular, a denominator  $2^m$  has a binary representation of length growing with  $m$ , and the finite rational powers in the unit-log series involve larger integers. The claim is elimination of an artificial need to increase *series depth* with the small scale, not constant-time arithmetic on arbitrarily large inputs.

## 8 Wolfram 15 and Mathics3 implications

### 8.1 What was verified on Wolfram

The main witnesses and edits were exercised on a real Wolfram 15.0.0 kernel, loading the complete downloaded standalone. The experiment therefore went beyond isolated helper transcription: public inverse objects were constructed and passed through the actual certificate driver. Both candidate text anchors were checked against the actual standalone, not merely against synthetic fixtures.

The observations still have specific limits. They are selected remote responses, not a complete MUnit log. The provided eight-test WLT file was not executed as one **TestReport**. The candidate emitter was locally tested on fixtures; equivalent substitutions were applied to the actual standalone with Wolfram string operations. Neither successful mechanism implies the canonical module was rebuilt and that its generated distribution was revalidated.

### 8.2 Why the source finding matters to Mathics

The compatibility documentation targets Mathics3 10.0.1 with scanner 10.0.1, SymPy 1.14.0, and Python 3.11. It explicitly distinguishes selected portable acceptance from complete compatibility. The Mathics adapters isolate missing or differing evaluator behavior in package-owned contexts rather than patching global System functions.[10, 13]

The logarithm certificate routines under review are shared exact-rational code, rather than the separate Mathics numerical logarithm-splitting guard. Their mathematical loss mechanisms do not depend on a machine floating-point sign test. The candidate uses rational comparison, reciprocal construction, interval negation, and the existing affine recognizer. These are appropriate operations for a portable repair, but that source-level observation is not an execution result.

There was no working Mathics kernel in this audit. Consequently the report makes no claim that the candidate loads, returns early correctly through the Mathics compatibility layer, meets its iteration



budget, or produces exactly the observed Wolfram matrix in Mathics. The supplied portable cases provide a concrete acceptance target for that missing run.

### 8.3 A focused cross-runtime acceptance obligation

The proposed integration should execute the same A/B/C requests on both runtimes and both supported layouts. Record whether each output is an association or failure before reading numerical fields. On a successful certificate, assert at least `Certified == True`, exact root containment, and a center error bound below  $\delta/8$ . This prevents a missing field or unevaluated expression from silently turning a numerical comparison into a vacuous success.

The first acceptance run should retain the same order and zero-refinement budget as the native evidence. Only afterward should adaptive defaults and larger scales be measured. This separates local arithmetic behavior from the already-existing driver's scheduling. The no-op exact-one, nonpositive-domain, positive-negative-factor, and nonlinear fallback controls should travel with the same patch.

The repository warns that a command-line expression parsed before package loading can bind unfamiliar public symbols in the wrong context. The supplied reproduction recipe therefore loads the package before the probe as separate input expressions, and the probe uses fully qualified public names. Its Mathics iteration-limit adjustment is explicit and matches the repository's guidance; it is not an unannounced package-global change. [10]

## 9 E01: center-preserving logarithmic residuals

### 9.1 A narrower next step than a general expression optimizer

F01 and F02 improve the logarithms supplied to the residual calculation. There is a further, optional opportunity: sometimes the exact relationship between the two logarithms in a residual can be retained instead of treating them independently.

For strictly positive real  $p$  and  $q$ ,

$$\log p - \log q = \log \left( \frac{p}{q} \right). \quad (13)$$

This is an elementary real identity, not a new mathematical result. Its application here should be a narrow, explicit certificate operation with positivity obligations, not a global principal-log simplification rule.

For  $p(x) = ax + b$  with rational coefficients and positive rational  $q$ , the exact affine image of  $p(x)/q$  can be passed to the logarithm routine. If an exact rational center  $c$  satisfies  $ac + b = q$ , the ratio at  $c$  is exactly one and its logarithm is exactly zero. Separate intervals for  $\log(ax + b)$  and  $\log q$  usually retain nonzero uncertainty even when both represent the same exact number.

**Proposition 9.1** (Centered affine log-ratio enclosure). *Let  $a, b, q, \ell, r \in \mathbb{Q}$ ,  $q > 0$ ,  $\ell \leq r$ , and  $ax + b > 0$  throughout  $[\ell, r]$ . Define*

$$J = \text{sort} \left\{ \frac{a\ell + b}{q}, \frac{ar + b}{q} \right\}.$$

*A containing logarithm interval for  $J$  contains  $\log(ax + b) - \log q$  throughout the source interval. At a singleton  $c$  with  $ac + b = q$ , the existing exact-one rule produces the exact residual interval  $[0, 0]$ .*



*Proof.* Division by positive  $q$  preserves the positive affine image. Equation (13) holds on that domain, and monotonicity of the logarithm then gives the enclosure. At the stated singleton the image is exactly  $\{1\}$ , whose logarithm is exactly zero.  $\square$

## 9.2 Executed primitive, unintegrated proposal

With the combined candidate loaded, the following primitive was executed natively and returned  $\{0,0\}$ :

```
certLogExpression[(1+x)/(1+2^-200), x,
  {2^-200, 2^-200}, ctx]
```

The call used the actual private helper under its package context. It does not establish that the public certificate dispatcher rewrites the original residual to this ratio. That integration was neither implemented nor claimed.

The archive's `code/LogRatioPrototype.wl` provides explicitly named rational and affine-ratio wrappers. The wrapper file itself was not executed as a separate Wolfram or Mathics program. Its corresponding centered primitive and the mathematical identity have the evidence stated above. It is a prototype for a narrowly guarded operation, not an installed repair.

## 9.3 Required guardrails

A future dispatcher integration should derive the ratio from retained positive-real data, not merely replace two syntactically matching `Log` nodes. Native evaluation may rewrite a negative logarithm or rearrange a rational target before the certificate routine sees it. A syntactic match that accidentally broadens to complex inputs would lose the real identity's hypotheses.

A zero point residual does not waive source-domain, continuity, derivative, or root-containment obligations. An exact root does not by itself prove uniqueness over the supplied interval or identify a global inverse branch. The existing certificate checks must remain in charge of those claims. The extension should preserve the original equation and identify any transformed residual in its evidence, so an error bound is never silently reinterpreted in another variable.

A sensible first implementation boundary is positive rational targets with rational affine logarithm arguments only. That boundary is sufficient to exercise exact centering, has an explicit proof, and avoids introducing a new general logarithmic normalization subsystem. More general positive expressions would require their own transformation and range contracts.

# 10 Tests, artifacts, and reproducibility

## 10.1 Independent test population

The independent Python run passed 24 unittest methods: five basic arithmetic methods, thirteen logarithm/model methods, and six candidate-emitter fixture methods. Some methods loop over many exact cases; those subcases are not inflated into a package acceptance count. The actual output is retained in `evidence/python-tests.txt`.

The exact model covers directed rounding for signed rationals, binary exponents, interval arithmetic, baseline and repaired point logarithms, affine transformations with positive and negative slopes, domain refusals, reciprocity, and the explicit unit-tail formula. Near-one families cover several small scales and

series orders. A separate mean-value-radius model checks a consequence of valid residual intervals; it is not presented as a transcription of the entire certificate driver.

High-precision mpmath checks are independent numerical containment controls, not proofs by themselves. One rational grid uses 180 decimal digits. The near-one controls use `log1p` with 450 digits, including  $\delta = 2^{-1000}$ . Construction of the rational bounds never uses the mpmath values or machine floating-point signs. The proofs in this article supply the general inclusion arguments; numerical controls help detect implementation mistakes in the independent model.

## 10.2 Candidate emitter and application discipline

The emitter accepts the canonical certificate module or a standalone file, checks that each selected anchor occurs exactly once, and writes a new candidate file. It refuses in-place replacement, an existing destination, missing or repeated anchors, and repeat application. Its local tests use synthetic anchor fixtures. This is useful fail-closed behavior, but not proof of correct integration into an arbitrary later source revision.

For example, from the archive root:

```
python code/emit_candidate.py \
/path/to/repo/AsymptoticAnalysis.wl \
/tmp/AsymptoticAnalysis.audit.wl --mode both
```

The other modes, `reciprocal` and `affine`, reproduce the separate columns of the experiment. To integrate, apply the selected edits in a staging checkout to `src/Kernel/InverseCertificates.wl`, regenerate the standalone by the repository's maintained process, and execute the focused acceptance there. This review did not perform that modular build.

## 10.3 Native and portable reproduction

Use separate input expressions after selecting a baseline or candidate path:

```
Get["/tmp/AsymptoticAnalysis.audit.wl"];
Get["/path/to/audit/code/PortableLogCases.wl"];
```

The portable file contains three public witnesses and primitive observations. Its expected baseline and candidate outcomes are the Wolfram-observed matrix, not asserted Mathics results. The eight desired-contract MUnit cases can be run on a native kernel after loading the combined candidate:

```
TestReport["/path/to/audit/code/ReviewLogPrecision.wlt"]
```

That complete WLT invocation was not executed in this audit. The archive retains this distinction even where its constituent cases correspond to successful individual native probes.

From `code/`, the independent checks run with:

```
python -m pip install -r requirements.txt
python -m unittest -v test_reference
```

The recorded Python environment was 3.13.5 with mpmath 1.3.0. This environment is independent of the repository’s documented Python 3.11 Mathics target; passing the Python model does not validate that target interpreter.

## 10.4 Evidence inventory

| Artifact                          | Meaning                                                                                                                                                          |
|-----------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| evidence/scope.json               | Commit, runtime, completed work, and explicit execution limits.                                                                                                  |
| evidence/findings.json            | Machine-readable descriptions of F01 and F02; no inflated bug or test totals.                                                                                    |
| evidence/native-matrix.json       | Selected transcription of the twelve controlled public evaluations. Displayed decimal mantissas do not imply more accuracy than the requested display precision. |
| evidence/native-observations.json | Individual primitive/public results, negative controls, and the numerical-display warning. Not a raw MUnit log.                                                  |
| evidence/python-tests.txt         | Actual output of the 24-method independent model and fixture run.                                                                                                |
| evidence/model-samples.csv        | Rational-model widths, rendered numerically for comparison; not measured kernel timings.                                                                         |
| evidence/order-cost-model.json    | Exact tail-threshold and grid-order models, including an explicitly theoretical large-scale row.                                                                 |
| evidence/novelty-crosswalk.md     | Nearest existing obligations and the limits of the completed nonduplication screen.                                                                              |

The source excerpts and anchors derive from the repository under its MIT No Attribution license. The archive preserves that license and a provenance notice. It does not redistribute the entire upstream package or its historical review corpus.[\[14\]](#)

## 11 Recommended integration sequence and conclusion

### 11.1 Immediate scoped work

First, integrate F01 and F02 as separately reviewable changes to the canonical certificate module. Each has a small local mathematical obligation and an individual witness. The combined gate must include witness C; otherwise a translated version of the same logarithmic equation can remain obstructed after a seemingly successful partial repair.

Second, run the supplied low-order cases in clean kernels on the intended Wolfram and Mathics versions, in modular and standalone layouts. Preserve exact source pinning and distinguish a script load failure, a structured refusal, an unevaluated result, and a certified association. The present native observations establish feasibility, not integration completion.

Third, measure only the changed paths at matched certificate goals. Include admitted affine logarithms, nonaffine fallbacks, arguments away from one, positive products with negative factors, and invalid domains. Report rational operand sizes and achieved root widths alongside runtime. This measurement should decide whether an extra failed affine probe is material; it should not weaken the exact-endpoint invariant merely to reduce a dispatch count.

Finally, treat E01 as a separate feature experiment. The centered-ratio primitive can preserve an exact point zero, but an integrated residual transformation requires positivity evidence, equation provenance, and all existing root checks. It should not delay the two smaller demonstrated repairs.

## 11.2 Final assessment

The newly identified weakness is not that the package uses floating-point arithmetic in its exact certificate proof. It does not in the primitives examined here. The weakness is that sound enclosure operations can discard valuable exact dependence and displacement information before the answer is formed. Two different interfaces inside the logarithm path do so, and the native matrix distinguishes them.

The repairs are correspondingly targeted: turn a below-one logarithm into one well-scaled positive reciprocal logarithm, and move affine endpoint rounding across the logarithm only when the exact affine range and positive real domain justify it. The resulting public successes retain the safety checks that caused the original refusals. This is a concrete improvement in certificate reach and resource use, with explicit proofs and executable regressions, rather than a proposal to accept weaker evidence.

The review's strongest conclusion is local and reproducible: on the pinned standalone in Wolfram 15.0.0, three exact-root requests that refused at the chosen fixed order certify after the combined changes, and every returned interval contains the known exact root. Complete Mathics acceptance and canonical-source integration remain to be demonstrated by the focused runs provided with this article.

## References

- [1] AsymptoticAnalysis contributors. *Asymptotic repository*, revision efa1aeec4845a9c35e140963a0333d0c9ec33b05, 10 September 2026. <https://github.com/VladimirReshetnikov/Asymptotic/tree/efa1aeec4845a9c35e140963a0333d0c9ec33b05>.
- [2] AsymptoticAnalysis contributors. *InverseCertificates.wl*, pinned canonical source. In particular, lines 8–18 (rounding), 54–72 (point logarithm), 80–130 (logarithm and affine range), 257–313 (certificate attempt), and 365–490 (driver). <https://github.com/VladimirReshetnikov/Asymptotic/blob/efa1aeec4845a9c35e140963a0333d0c9ec33b05/src/Kernel/InverseCertificates.wl>.
- [3] AsymptoticAnalysis contributors. *AsymptoticAnalysis.wl*, canonical kernel entry point and forward jet arithmetic, same revision. <https://github.com/VladimirReshetnikov/Asymptotic/blob/efa1aeec4845a9c35e140963a0333d0c9ec33b05/src/Kernel/AsymptoticAnalysis.wl>.
- [4] AsymptoticAnalysis contributors. *ParameterizedSpecialFunctions.wl*, same revision. <https://github.com/VladimirReshetnikov/Asymptotic/blob/efa1aeec4845a9c35e140963a0333d0c9ec33b05/src/Kernel/ParameterizedSpecialFunctions.wl>.
- [5] AsymptoticAnalysis contributors. *SpecialFunctionIdentities.wl*, same revision. <https://github.com/VladimirReshetnikov/Asymptotic/blob/efa1aeec4845a9c35e140963a0333d0c9ec33b05/src/Kernel/SpecialFunctionIdentities.wl>.
- [6] AsymptoticAnalysis contributors. *Code review reports: index, six wave indexes, and retirement register*, same revision. Forty-five retained packages. <https://github.com/VladimirReshetnikov/Asymptotic/blob/efa1aeec4845a9c35e140963a0333d0c9ec33b05/external-reports/code-review/README.md>.
- [7] AsymptoticAnalysis contributors. *Code review implementation status*, same revision. C19, C20, and P08 are the nearest maintained obligations discussed in the novelty crosswalk. [https://github.com/VladimirReshetnikov/Asymptotic/blob/efa1aeec4845a9c35e140963a0333d0c9ec33b05/docs/development/CODE\\_REVIEW\\_STATUS.md](https://github.com/VladimirReshetnikov/Asymptotic/blob/efa1aeec4845a9c35e140963a0333d0c9ec33b05/docs/development/CODE_REVIEW_STATUS.md).
- [8] AsymptoticAnalysis contributors. *Code review reports: wave 5*, same revision. <https://github.com/VladimirReshetnikov/Asymptotic/blob/efa1aeec4845a9c35e140963a0333d0c9ec33b05/external-reports/code-review/wave-5/README.md>.
- [9] AsymptoticAnalysis contributors. *Code review reports: wave 6*, same revision. Reports 46–55, including its explicit overlap and evidence-boundary records. <https://github.com/VladimirReshetnikov/Asymptotic/blob/efa1aeec4845a9c35e140963a0333d0c9ec33b05/external-reports/code-review/wave-6/README.md>.
- [10] AsymptoticAnalysis contributors. *Running AsymptoticAnalysis in Mathics3*, pinned compatibility guide. <https://github.com/VladimirReshetnikov/Asymptotic/blob/efa1aeec4845a9c35e140963a0333d0c9ec33b05/docs/Mathics/COMPATIBILITY.md>.
- [11] NIST Digital Library of Mathematical Functions. *Section 4.6: Power Series*, especially equation 4.6.4 for the odd logarithmic series. Consulted 10 September 2026. <https://dlmf.nist.gov/4.6>.
- [12] Wolfram Research. *Log: Wolfram Language documentation*. Background on logarithm semantics and numerical evaluation; consulted 10 September 2026. <https://reference.wolfram.com/language/ref/Log.html>.
- [13] Mathics3 project. *Mathics3 official website*. Consulted 10 September 2026. The repository compatibility guide, not this general website, supplies the specific tested dependency target used in this report. <https://mathics.org/>.
- [14] AsymptoticAnalysis contributors. *MIT No Attribution License*, pinned repository license. <https://github.com/VladimirReshetnikov/Asymptotic/blob/efa1aeec4845a9c35e140963a0333d0c9ec33b05/LICENSE>.